



Faculty of Engineering
Data Structures and Algorithms
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: BRANCH:

SEMESTER: ACADEMIC YEAR:



Swaminarayan university
Publication

ISBN Number: 978-81-998196-8-9



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in has satisfactorily completed his
term working (Subject name) (subject code:)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASHWINI WALAKE
SUBJECT IN-CHARGE

PROF. NIMESH VAIDHYA
HOD

DR. BHAVNA SONI
DEAN

List of Practicals

Sr. No.	Title	Page No.	Date	Sign
1	Define various terms such as algorithm, various approaches to design an algorithm, time complexity, space complexity, best case, average case and worst-case time complexity etc.			
2	Implement array using row major order and column major order			
3	Implement Sequential search algorithms.			
4	Implement Binary search algorithms.			
5	Implement various string algorithms.			
6	Implement push and pop algorithms of stack using array			
7	Implement recursive functions.			
8	Implement insert algorithms of queue using array			
9	Implement delete algorithms of queue using array.			
10	Implement simple structure programs using pointers			
11	Implement insertion of node in the beginning of the list in singly linked list.			

12	Implement insertion of node at the end of list in singly linked list.			
13	Implement insertion of node in sorted linked list.			
14	Implement insertion of node at any position in linked list.			
15	Implement counting no of node algorithm in singly linked list.			
16	Implement searching of a node algorithm in singly linked list.			
17	Implement delete a node algorithm in singly linked list			
18	Implement construction of binary search tree.			
19	Implement searching algorithm in binary search tree.			
20	Implement Inorder, preorder and postorder traversal methods in binary search tree.			
21	Implement Bubble sort algorithm.			
22	Implement Quick Sort algorithm.			
23	Implement Insertion sort algorithm.			
24	Implement Shell sort algorithm.			
25	Implement Selection sort algorithm.			
26	Implement Merge Sort algorithm.			

Practical-1

Aim: Define various terms such as algorithm, various approaches to design an algorithm, time complexity, space complexity, best case, average case and worst-case time complexity etc.

```
#include <stdio.h>

int main() {
    int arr[100], n, key;
    int found = 0;

    // Input size of array
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Input array elements
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Input element to search
    printf("Enter element to search: ");
    scanf("%d", &key);

    // Linear Search
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            printf("Element found at index %d\n", i);
            found = 1;
            break; // stop when found (best-case behavior)
        }
    }

    if (!found)
        printf("Element not found\n");
    return 0;
}
```

Output

```
Enter the number of elements: 5
Enter 5 elements:
10 20 30 40 50
Enter element to search: 30
Element found at index 2
```

Practical-2

Aim: Implement array using row major order and column major order

```
#include <stdio.h>

int main() {
    int m, n;

    // Taking the number of rows and columns
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &m, &n);

    int arr[m][n];

    // Taking input for the matrix
    printf("Enter elements of the matrix:\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            printf("Enter element at (%d,%d): ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    // Displaying matrix in row-major order
    printf("\nMatrix in Row-Major Order:\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", arr[i][j]);
        }
    }

    // Displaying matrix in column-major order
    printf("\nMatrix in Column-Major Order:\n");
    for (int j = 0; j < n; j++) {
        for (int i = 0; i < m; i++) {
            printf("%d ", arr[i][j]);
        }
    }

    return 0;
}
```

Output

Enter the number of rows and columns: 2 3

Enter elements of the matrix:

Enter element at (0,0): 1

Enter element at (0,1): 2

Enter element at (0,2): 3

Enter element at (1,0): 4

Enter element at (1,1): 5

Enter element at (1,2): 6

Matrix in Row-Major Order:

1 2 3 4 5 6

Matrix in Column-Major Order:

1 4 2 5 3 6

Practical-3

Aim: Implement Sequential search algorithms.

```
#include <stdio.h>

// Function for Sequential Search
int sequentialSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i; // Return index if element found
    }
    return -1; // Return -1 if element not found
}

int main() {
    int n, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);

    int result = sequentialSearch(arr, n, key);

    if (result == -1)
        printf("Element %d not found in the array.\n", key);
    else
        printf("Element %d found at position %d (index %d).\n", key, result + 1, result);

    return 0;
}
```

Output

Enter number of elements: 5

Enter 5 elements:

10 20 30 40 50

Enter the element to search: 30

Element 30 found at position 3 (index 2).

Practical-4

Aim: Implement Binary search algorithms.

```
#include <stdio.h>

// Function to perform Binary Search
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid; // Found the key at mid index
        else if (arr[mid] < key)
            low = mid + 1; // Search in right half
        else
            high = mid - 1; // Search in left half
    }

    return -1; // Not found
}

int main() {
    int n, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements in sorted order:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);

    int result = binarySearch(arr, n, key);

    if (result == -1)
        printf("Element %d not found in the array.\n", key);
    else
```

```
    printf("Element %d found at position %d (index %d).\n", key, result + 1, result);  
    return 0;  
}
```

Output

Enter number of elements: 6

Enter 6 elements in sorted order:

5 10 15 20 25 30

Enter the element to search: 20

Element 20 found at position 4 (index 3).

Practical-5

Aim: Implement various string algorithms.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

// 1. String Length
int stringLength(char str[]) {
    int len = 0;
    while (str[len] != '\0') {
        len++;
    }
    return len;
}

// 2. String Copy
void stringCopy(char dest[], char src[]) {
    int i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}

// 3. String Concatenation
void stringConcat(char str1[], char str2[]) {
    int i = 0, j = 0;
    while (str1[i] != '\0') {
        i++;
    }
    while (str2[j] != '\0') {
        str1[i++] = str2[j++];
    }
    str1[i] = '\0';
}

// 4. String Comparison
int stringCompare(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] != '\0' && str2[i] != '\0') {
        if (str1[i] != str2[i]) {
```

```
        return (str1[i] - str2[i]);
    }
    i++;
}
return (str1[i] - str2[i]);
}
```

// 5. Substring Search (Naive)

```
int substringSearch(char text[], char pattern[]) {
    int n = stringLength(text);
    int m = stringLength(pattern);

    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m)
            return i; // found at index i
    }
    return -1; // not found
}
```

// 6. Palindrome Check

```
bool isPalindrome(char str[]) {
    int start = 0;
    int end = stringLength(str) - 1;
    while (start < end) {
        if (str[start] != str[end])
            return false;
        start++;
        end--;
    }
    return true;
}
```

```
int main() {
    char str1[100] = "Hello";
    char str2[100] = "World";
    char copy[100];

    printf("Length of '%s' = %d\n", str1, stringLength(str1));

    stringCopy(copy, str1);
}
```

```
printf("Copied String: %s\n", copy);

stringConcat(str1, str2);
printf("Concatenated String: %s\n", str1);

int cmp = stringCompare("apple", "apricot");
if (cmp == 0)
    printf("Strings are equal\n");
else if (cmp < 0)
    printf("\"apple\" is less than \"apricot\"\n");
else
    printf("\"apple\" is greater than \"apricot\"\n");

char text[] = "datastructures";
char pattern[] = "struct";
int index = substringSearch(text, pattern);
if (index != -1)
    printf("Pattern \"%s\" found at index %d in \"%s\"\n", pattern, index, text);
else
    printf("Pattern not found\n");

char pal[] = "madam";
if (isPalindrome(pal))
    printf("\"%s\" is a palindrome\n", pal);
else
    printf("\"%s\" is not a palindrome\n", pal);

return 0;
}
```

Output

```
Length of 'Hello' = 5
Copied String: Hello
Concatenated String: HelloWorld
"apple" is less than "apricot"
Pattern "struct" found at index 4 in "datastructures"
"madam" is a palindrome
```

Practical-6

Aim: Implement push and pop algorithms of stack using array

```
#include <stdio.h>
#define MAX 5 // Maximum size of stack

int stack[MAX];
int top = -1; // Empty stack initially

// Function to push an element
void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow! Cannot push %d\n", value);
    } else {
        top++;
        stack[top] = value;
        printf("%d pushed into stack\n", value);
    }
}

// Function to pop an element
void pop() {
    if (top == -1) {
        printf("Stack Underflow! Cannot pop\n");
    } else {
        printf("%d popped from stack\n", stack[top]);
        top--;
    }
}

// Function to display stack contents
void display() {
    if (top == -1) {
        printf("Stack is empty\n");
    } else {
        printf("Stack elements: ");
        for (int i = 0; i <= top; i++) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
```

```
int main() {  
    push(10);  
    push(20);  
    push(30);  
    display();  
  
    pop();  
    display();  
  
    push(40);  
    push(50);  
    push(60); // this will show overflow  
    display();  
  
    return 0;  
}
```

Output

```
10 pushed into stack  
20 pushed into stack  
30 pushed into stack  
Stack elements: 10 20 30  
30 popped from stack  
Stack elements: 10 20  
40 pushed into stack  
50 pushed into stack  
Stack Overflow! Cannot push 60  
Stack elements: 10 20 40 50
```

Practical-7

Aim: Implement recursive functions.

```
#include <stdio.h>

// Recursive function to calculate factorial
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1; // Base case
    else
        return n * factorial(n - 1); // Recursive call
}

// Recursive function to find nth Fibonacci number
int fibonacci(int n) {
    if (n == 0)
        return 0; // Base case
    else if (n == 1)
        return 1; // Base case
    else
        return fibonacci(n - 1) + fibonacci(n - 2); // Recursive calls
}

// Recursive function to find sum of array elements
int sumArray(int arr[], int n) {
    if (n == 0)
        return 0; // Base case
    else
        return arr[n - 1] + sumArray(arr, n - 1); // Recursive call
}

int main() {
    int n;

    // Factorial example
    n = 5;
    printf("Factorial of %d = %d\n", n, factorial(n));

    // Fibonacci example
    n = 6;
    printf("Fibonacci series up to %d terms: ", n);
    for (int i = 0; i < n; i++)
        printf("%d ", fibonacci(i));
```

```
printf("\n");

// Sum of array example
int arr[] = {1, 2, 3, 4, 5};
int size = sizeof(arr) / sizeof(arr[0]);
printf("Sum of array elements = %d\n", sumArray(arr, size));

return 0;
}
```

Output

Factorial of 5 = 120

Fibonacci series up to 6 terms: 0 1 1 2 3 5

Sum of array elements = 15

Practical-8

Aim: Implement insert algorithms of queue using array.

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Function to insert (enqueue) an element into the queue
void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow! Cannot insert %d\n", value);
        return;
    }
    if (front == -1) // first element
        front = 0;

    rear++;
    queue[rear] = value;
    printf("Inserted %d into the queue\n", value);
}

// Function to display the queue
void display() {
    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    // Inserting elements into the queue
    enqueue(10);
    enqueue(20);
    enqueue(30);

    // Display the queue after insertions
    display();
}
```

```
enqueue(40);
enqueue(50);
display();

// Trying to insert when queue is full
enqueue(60);

return 0;
}
```

Output

Inserted 10 into the queue
Inserted 20 into the queue
Inserted 30 into the queue
Queue elements: 10 20 30
Inserted 40 into the queue
Inserted 50 into the queue
Queue elements: 10 20 30 40 50
Queue Overflow! Cannot insert 60

Practical-9

Aim: Implement delete algorithms of queue using array.

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Function to insert (enqueue) an element into the queue
void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow! Cannot insert %d\n", value);
        return;
    }
    if (front == -1) // First element being inserted
        front = 0;

    rear++;
    queue[rear] = value;
    printf("Inserted %d into the queue\n", value);
}

// Function to delete (dequeue) an element from the queue
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow! Cannot delete.\n");
        return;
    }
    printf("Deleted %d from the queue\n", queue[front]);
    front++;
}

// Function to display the queue elements
void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
}
```

```
    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();

    dequeue();
    display();

    dequeue();
    dequeue();
    display();

    dequeue(); // Trying to delete from an empty queue

    return 0;
}
```

Output

Inserted 10 into the queue
Inserted 20 into the queue
Inserted 30 into the queue
Queue elements: 10 20 30
Deleted 10 from the queue
Queue elements: 20 30
Deleted 20 from the queue
Deleted 30 from the queue
Queue is empty
Queue Underflow! Cannot delete.

Practical-10

Aim: Implement simple structure programs using pointers.

```
#include <stdio.h>
#include <string.h>

// Define a structure
struct Student {
    int roll_no;
    char name[50];
    float marks;
};

int main() {
    struct Student s1;      // Declare structure variable
    struct Student *ptr;     // Declare pointer to structure

    ptr = &s1;              // Assign address of s1 to pointer

    // Input data using pointer
    printf("Enter Roll Number: ");
    scanf("%d", &ptr->roll_no);

    printf("Enter Name: ");
    scanf("%s", ptr->name);

    printf("Enter Marks: ");
    scanf("%f", &ptr->marks);

    // Display data using pointer
    printf("\n--- Student Details ---\n");
    printf("Roll Number: %d\n", ptr->roll_no);
    printf("Name: %s\n", ptr->name);
    printf("Marks: %.2f\n", ptr->marks);

    return 0;
}
```

Output

```
Enter Roll Number: 101
Enter Name: Ishani
Enter Marks: 88.5
```

--- Student Details ---

Roll Number: 101

Name: Ishani

Marks: 88.50

Practical-11

Aim: Implement insertion of node in the beginning of the list in singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Function to insert a new node at the beginning
void insertAtBeginning(struct Node **head, int value) {
    // Allocate memory for new node
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;

    // Point new node's next to current head
    newNode->next = *head;

    // Make new node as the head
    *head = newNode;

    printf("%d inserted at the beginning\n", value);
}

// Function to display the linked list
void display(struct Node *head) {
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head = NULL; // Start with an empty list
    int n, value;

    printf("How many nodes you want to insert at beginning? ");
```

```
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    printf("Enter value for node %d: ", i + 1);
    scanf("%d", &value);
    insertAtBeginning(&head, value);
    display(head); // Show list after each insertion
}

return 0;
}
```

Output

How many nodes you want to insert at beginning? 3
Enter value for node 1: 10
10 inserted at the beginning
Linked List: 10 -> NULL
Enter value for node 2: 20
20 inserted at the beginning
Linked List: 20 -> 10 -> NULL
Enter value for node 3: 30
30 inserted at the beginning
Linked List: 30 -> 20 -> 10 -> NULL

Practical-12

Aim: Implement insertion of node at the end of list in singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Function to insert a node at the end of the list
void insertAtEnd(struct Node **head, int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (*head == NULL) {
        // If list is empty, new node becomes head
        *head = newNode;
    } else {
        struct Node *temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }

    printf("%d inserted at the end\n", value);
}

// Function to display the linked list
void display(struct Node *head) {
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
int main() {
    struct Node *head = NULL; // Empty list initially
    int n, value;

    printf("How many nodes you want to insert at the end? ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value for node %d: ", i + 1);
        scanf("%d", &value);
        insertAtEnd(&head, value);
        display(head); // Display after each insertion
    }

    return 0;
}
```

Output

How many nodes you want to insert at the end? 3
Enter value for node 1: 10
10 inserted at the end
Linked List: 10 -> NULL
Enter value for node 2: 20
20 inserted at the end
Linked List: 10 -> 20 -> NULL
Enter value for node 3: 30
30 inserted at the end
Linked List: 10 -> 20 -> 30 -> NULL

Practical-13

Aim: Implement insertion of node in sorted linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define structure for a node
struct Node {
    int data;
    struct Node *next;
};

// Function to insert a node in a sorted linked list
void insertSorted(struct Node **head, int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    // If list is empty or new node should be inserted at the beginning
    if (*head == NULL || (*head)->data >= value) {
        newNode->next = *head;
        *head = newNode;
    } else {
        struct Node *temp = *head;
        // Find position to insert
        while (temp->next != NULL && temp->next->data < value) {
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }

    printf("%d inserted into sorted list.\n", value);
}

// Function to display the linked list
void display(struct Node *head) {
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
int main() {
    struct Node *head = NULL; // Initially empty
    int n, value;

    printf("How many nodes you want to insert in sorted order? ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value for node %d: ", i + 1);
        scanf("%d", &value);
        insertSorted(&head, value);
        display(head); // Display after each insertion
    }

    return 0;
}
```

Output

How many nodes you want to insert in sorted order? 5
Enter value for node 1: 30
30 inserted into sorted list.
Linked List: 30 -> NULL
Enter value for node 2: 10
10 inserted into sorted list.
Linked List: 10 -> 30 -> NULL
Enter value for node 3: 50
50 inserted into sorted list.
Linked List: 10 -> 30 -> 50 -> NULL
Enter value for node 4: 20
20 inserted into sorted list.
Linked List: 10 -> 20 -> 30 -> 50 -> NULL
Enter value for node 5: 40
40 inserted into sorted list.
Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

Practical-14

Aim: Implement insertion of node at any position in linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define structure of node
struct Node {
    int data;
    struct Node *next;
};

// Function to insert node at any position
void insertAtPosition(struct Node **head, int data, int position) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    // If inserting at the beginning
    if (position == 1) {
        newNode->next = *head;
        *head = newNode;
        printf("%d inserted at position %d.\n", data, position);
        return;
    }

    struct Node *temp = *head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position %d is out of range.\n", position);
        free(newNode);
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    printf("%d inserted at position %d.\n", data, position);
}
```

```
// Function to display linked list
void display(struct Node *head) {
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head = NULL;
    int n, value, pos;

    printf("How many nodes do you want to insert? ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nEnter value for node %d: ", i + 1);
        scanf("%d", &value);
        printf("Enter position to insert this node: ");
        scanf("%d", &pos);

        insertAtPosition(&head, value, pos);
        display(head);
    }

    return 0;
}
```

Output

How many nodes do you want to insert? 4

Enter value for node 1: 10
Enter position to insert this node: 1
10 inserted at position 1.
Linked List: 10 -> NULL

Enter value for node 2: 20
Enter position to insert this node: 2
20 inserted at position 2.
Linked List: 10 -> 20 -> NULL

Enter value for node 3: 15

Enter position to insert this node: 2
15 inserted at position 2.
Linked List: 10 -> 15 -> 20 -> NULL

Enter value for node 4: 5
Enter position to insert this node: 1
5 inserted at position 1.
Linked List: 5 -> 10 -> 15 -> 20 -> NULL

Practical-15

Aim: Implement counting no of node algorithm in singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define a node
struct Node {
    int data;
    struct Node *next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert node at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Function to count number of nodes
int countNodes(struct Node* head) {
    int count = 0;
    struct Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

```
// Function to display list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    // Inserting nodes
    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 30);
    insertEnd(&head, 40);

    printf("Linked List: ");
    display(head);

    int total = countNodes(head);
    printf("Total number of nodes = %d\n", total);

    return 0;
}
```

Output

Linked List: 10 -> 20 -> 30 -> 40 -> NULL
Total number of nodes = 4

Practical-16

Aim: Implement searching of a node algorithm in singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define a node
struct Node {
    int data;
    struct Node *next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert node at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

// Function to search a node
int searchNode(struct Node* head, int key) {
    int position = 1;
    struct Node* temp = head;

    while (temp != NULL) {
        if (temp->data == key)
            return position; // return position if found
        temp = temp->next;
        position++;
    }
}
```

```
    }
    return -1; // not found
}

// Function to display list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    // Insert some nodes
    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 30);
    insertEnd(&head, 40);

    printf("Linked List: ");
    display(head);

    int key = 30;
    int result = searchNode(head, key);

    if (result != -1)
        printf("Node with value %d found at position %d\n", key, result);
    else
        printf("Node with value %d not found\n", key);

    return 0;
}
```

Output

Linked List: 10 -> 20 -> 30 -> 40 -> NULL
Node with value 30 found at position 3

Practical-17

Aim: Implement delete a node algorithm in singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

// Define the node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert node at the end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Function to delete a node by key
void deleteNode(struct Node** head, int key) {
    struct Node* temp = *head;
    struct Node* prev = NULL;

    // If head node itself holds the key
    if (temp != NULL && temp->data == key) {
        *head = temp->next; // Change head
        free(temp);        // Free old head
    }
```

```
    printf("Node with value %d deleted.\n", key);
    return;
}

// Search for the key to be deleted
while (temp != NULL && temp->data != key) {
    prev = temp;
    temp = temp->next;
}

// If key was not found
if (temp == NULL) {
    printf("Node with value %d not found.\n", key);
    return;
}

// Unlink the node and free memory
prev->next = temp->next;
free(temp);
printf("Node with value %d deleted.\n", key);
}

// Function to print the linked list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

    // Insert some nodes
    insertEnd(&head, 10);
    insertEnd(&head, 20);
    insertEnd(&head, 30);
    insertEnd(&head, 40);

    printf("Original list:\n");
    printList(head);
```

```
// Delete a node
deleteNode(&head, 20); // Delete middle node
printf("After deletion:\n");
printList(head);

deleteNode(&head, 10); // Delete first node
printf("After deletion of head:\n");
printList(head);

deleteNode(&head, 50); // Node not in list
printf("After attempting to delete non-existent node:\n");
printList(head);

return 0;
}
```

Output

Original list:

10 -> 20 -> 30 -> 40 -> NULL

Node with value 20 deleted.

After deletion:

10 -> 30 -> 40 -> NULL

Node with value 10 deleted.

After deletion of head:

30 -> 40 -> NULL

Node with value 50 not found.

After attempting to delete non-existent node:

30 -> 40 -> NULL

Practical-18

Aim: Implement construction of binary search tree.

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure of a node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Function to insert a node into the BST
struct Node* insertBST(struct Node* root, int data) {
    // If tree is empty, create root
    if (root == NULL) {
        return createNode(data);
    }

    // Otherwise, recur down the tree
    if (data < root->data) {
        root->left = insertBST(root->left, data);
    } else if (data > root->data) {
        root->right = insertBST(root->right, data);
    }

    // Duplicate values are not inserted in BST
    return root;
}

// Function for inorder traversal (Left, Root, Right)
void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
    }
}
```

```
        inorderTraversal(root->right);
    }
}

// Function for preorder traversal (Root, Left, Right)
void preorderTraversal(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

// Function for postorder traversal (Left, Right, Root)
void postorderTraversal(struct Node* root) {
    if (root != NULL) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}

// Main function
int main() {
    struct Node* root = NULL;

    // Insert nodes into BST
    root = insertBST(root, 50);
    insertBST(root, 30);
    insertBST(root, 70);
    insertBST(root, 20);
    insertBST(root, 40);
    insertBST(root, 60);
    insertBST(root, 80);

    // Display traversals
    printf("Inorder Traversal (Sorted order):\n");
    inorderTraversal(root);
    printf("\n");

    printf("Preorder Traversal:\n");
    preorderTraversal(root);
    printf("\n");

    printf("Postorder Traversal:\n");
```

```
    postorderTraversal(root);  
    printf("\n");  
  
    return 0;  
}
```

Output

Inorder Traversal (Sorted order):

20 30 40 50 60 70 80

Preorder Traversal:

50 30 20 40 70 60 80

Postorder Traversal:

20 40 30 60 80 70 50

Practical-19

Aim: Implement searching algorithm in binary search tree.

```
#include <stdio.h>
#include <stdlib.h>

// Structure for each node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert a node into BST
struct Node* insertBST(struct Node* root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data)
        root->left = insertBST(root->left, data);
    else if (data > root->data)
        root->right = insertBST(root->right, data);

    return root;
}

// Search for a key in BST
struct Node* searchBST(struct Node* root, int key) {
    if (root == NULL || root->data == key)
        return root; // either found or tree is empty

    if (key < root->data)
        return searchBST(root->left, key);
}
```

```
    return searchBST(root->right, key);
}

// Inorder traversal (for displaying the tree)
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Main function
int main() {
    struct Node* root = NULL;

    // Construct BST
    root = insertBST(root, 50);
    insertBST(root, 30);
    insertBST(root, 70);
    insertBST(root, 20);
    insertBST(root, 40);
    insertBST(root, 60);
    insertBST(root, 80);

    printf("Inorder traversal of the BST:\n");
    inorder(root);
    printf("\n");

    int key = 60;
    struct Node* result = searchBST(root, key);
    if (result != NULL)
        printf("Node with value %d found in BST.\n", key);
    else
        printf("Node with value %d not found in BST.\n", key);

    key = 25;
    result = searchBST(root, key);
    if (result != NULL)
        printf("Node with value %d found in BST.\n", key);
    else
        printf("Node with value %d not found in BST.\n", key);

    return 0;
}
```

Output

Inorder traversal of the BST:

20 30 40 50 60 70 80

Node with value 60 found in BST.

Node with value 25 not found in BST.

Practical-20

Aim: Implement Inorder, preorder and postorder traversal methods in binary search tree.

```
#include <stdio.h>
#include <stdlib.h>

// Structure for each node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert a node into the BST
struct Node* insertBST(struct Node* root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data)
        root->left = insertBST(root->left, data);
    else if (data > root->data)
        root->right = insertBST(root->right, data);

    return root;
}

// Inorder Traversal: Left → Root → Right
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
// Preorder Traversal: Root → Left → Right
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder Traversal: Left → Right → Root
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// Main function
int main() {
    struct Node* root = NULL;

    // Construct the BST
    root = insertBST(root, 50);
    insertBST(root, 30);
    insertBST(root, 70);
    insertBST(root, 20);
    insertBST(root, 40);
    insertBST(root, 60);
    insertBST(root, 80);

    printf("Inorder Traversal (Left, Root, Right):\n");
    inorder(root);
    printf("\n");

    printf("Preorder Traversal (Root, Left, Right):\n");
    preorder(root);
    printf("\n");

    printf("Postorder Traversal (Left, Right, Root):\n");
    postorder(root);
    printf("\n");

    return 0;
}
```

```
}
```

Output

Inorder Traversal (Left, Root, Right):

20 30 40 50 60 70 80

Preorder Traversal (Root, Left, Right):

50 30 20 40 70 60 80

Postorder Traversal (Left, Right, Root):

20 40 30 60 80 70 50

Practical-21

Aim: Implement Bubble sort algorithm.

```
#include <stdio.h>

// Function to perform Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0; // Optimization: track if any swap happens
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap adjacent elements if they are in wrong order
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        // If no swaps happened, array is already sorted
        if (!swapped)
            break;
    }
}

// Function to print an array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Main function
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array:\n");
    printArray(arr, n);

    bubbleSort(arr, n);

    printf("Sorted array (using Bubble Sort):\n");
```

```
    printArray(arr, n);  
    return 0;  
}
```

Output

Original array:

64 25 12 22 11

Sorted array (using Bubble Sort):

11 12 22 25 64

Practical-22

Aim: Implement Quick Sort algorithm.

```
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // choosing last element as pivot
    int i = (low - 1);      // index of smaller element

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // partition index
        int pi = partition(arr, low, high);

        // Recursively sort elements before and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print an array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

```
    }  
    printf("\n");  
}  
  
// Main function  
int main() {  
    int arr[] = {64, 25, 12, 22, 11};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Original array:\n");  
    printArray(arr, n);  
  
    quickSort(arr, 0, n - 1);  
  
    printf("Sorted array (using Quick Sort):\n");  
    printArray(arr, n);  
  
    return 0;  
}
```

Output

Original array:

64 25 12 22 11

Sorted array (using Quick Sort):

11 12 22 25 64

Practical-23

Aim: Implement Insertion sort algorithm.

```
#include <stdio.h>

// Function to perform Insertion Sort
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];    // current element to insert
        int j = i - 1;

        // Shift elements of sorted part that are greater than key
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key; // insert key at correct position
    }
}

// Function to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Main function
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array:\n");
    printArray(arr, n);

    insertionSort(arr, n);

    printf("Sorted array (using Insertion Sort):\n");
    printArray(arr, n);

    return 0;
}
```

```
}
```

Output

Original array:

64 25 12 22 11

Sorted array (using Insertion Sort):

11 12 22 25 64

Practical-24

Aim: Implement Shell sort algorithm.

```
#include <stdio.h>

// Function to perform Shell Sort
void shellSort(int arr[], int n) {
    // Start with a big gap, then reduce the gap
    for (int gap = n / 2; gap > 0; gap /= 2) {
        // Do a gapped insertion sort for this gap size
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j = i;

            // Shift earlier gap-sorted elements up until the correct location for arr[i] is found
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }
}

// Function to print an array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Main function
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array:\n");
    printArray(arr, n);

    shellSort(arr, n);
}
```

```
printf("Sorted array (using Shell Sort):\n");  
printArray(arr, n);  
  
return 0;  
}
```

Output

Original array:

64 25 12 22 11

Sorted array (using Shell Sort):

11 12 22 25 64

Practical-25

Aim: Implement Selection sort algorithm.

```
#include <stdio.h>

// Function to perform Selection Sort
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find the index of the minimum element in the unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }

        // Swap the found minimum element with the first element of unsorted part
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

// Function to print an array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Main function
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array:\n");
    printArray(arr, n);

    selectionSort(arr, n);
}
```

```
printf("Sorted array (using Selection Sort):\n");  
printArray(arr, n);  
  
return 0;  
}
```

Output

Original array:

64 25 12 22 11

Sorted array (using Selection Sort):

11 12 22 25 64

Element found at index 2

Practical-26

Aim: Implement Merge Sort algorithm.

```
#include <stdio.h>

// Function to merge two subarrays of arr[]
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1; // size of left subarray
    int n2 = right - mid;    // size of right subarray

    int L[n1], R[n2];

    // Copy data to temp arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Merge the temp arrays back into arr[left..right]
    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements of L[]
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of R[]
    while (j < n2) {
        arr[k] = R[j];
        j++;
    }
}
```

```
        k++;
    }
}

// Function to implement merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Function to print the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

Output

Original array: 38 27 43 3 9 82 10
Sorted array: 3 9 10 27 38 43 82

